

# **MODUL PRAKTIKUM**

**Prediksi Awal Kesehatan Daun Berbasis Citra Digital**

**PROGRAM STUDI  
INSTITUSI PENDIDIKAN  
2024**

## Kata Pengantar

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulisan "Modul Praktikum Prediksi Awal Kesehatan Daun Berbasis Citra Digital" ini dapat diselesaikan dengan baik.

Modul ini dirancang secara khusus untuk memberikan pengalaman praktis secara komprehensif dari awal hingga akhir (end-to-end) dalam membangun sebuah sistem Machine Learning berbasis Computer Vision. Melalui studi kasus klasifikasi kondisi daun menjadi 5 (lima) kelas, pembaca akan dipandu mulai dari tahapan pemuatan data, eksplorasi data, perancangan model jaringan saraf tiruan (ResNet9), pelatihan model, hingga tahapan *deployment* menjadi sebuah aplikasi web sederhana menggunakan Streamlit.

Penggunaan Google Colab sebagai environment utama dalam praktikum ini diharapkan dapat memudahkan pembaca dalam mempraktekkan materi tanpa perlu memikirkan spesifikasi perangkat keras yang tinggi, karena seluruh komputasi dilakukan melalui teknologi *cloud*.

Kami menyadari bahwa modul ini tidak luput dari kekurangan. Oleh karena itu, segala bentuk kritik dan saran yang membangun sangat kami harapkan guna penyempurnaan di masa yang akan datang. Perlu ditegaskan pula bahwa model yang dibangun dalam modul ini berfungsi sebagai purwarupa (prototype) untuk *screening* awal, dan bukan sebagai diagnosis agrikultural yang final.

Akhir kata, semoga modul ini dapat memberikan manfaat yang sebesar-besarnya bagi para pembaca, khususnya bagi pihak yang sedang mendalami implementasi Machine Learning dan Computer Vision di dunia nyata.

Penyusun

## Daftar Isi

|                                                       |          |
|-------------------------------------------------------|----------|
| <b>Kata Pengantar</b>                                 | <b>i</b> |
| <b>1 Pendahuluan</b>                                  | <b>1</b> |
| 1.1 Gambaran Umum . . . . .                           | 1        |
| 1.2 Tujuan Praktikum . . . . .                        | 2        |
| 1.3 Batasan Sistem . . . . .                          | 2        |
| 1.4 Persiapan Lingkungan Praktikum . . . . .          | 2        |
| <b>2 Persiapan dan Eksplorasi Data</b>                | <b>4</b> |
| 2.1 Data Loading dengan KaggleHub . . . . .           | 4        |
| 2.1.1 Pengambilan Data . . . . .                      | 4        |
| 2.1.2 Mengecek Struktur Folder Dataset . . . . .      | 5        |
| 2.2 Persiapan Environment . . . . .                   | 6        |
| 2.2.1 Mengimpor Library . . . . .                     | 6        |
| 2.2.2 Menginstal Library Tambahan . . . . .           | 7        |
| 2.3 Exploratory Data Analysis (EDA) . . . . .         | 7        |
| 2.3.1 Menentukan Lokasi Folder Dataset . . . . .      | 8        |
| 2.3.2 Menghitung Total Gambar dalam Dataset . . . . . | 9        |
| 2.3.3 Menampilkan Contoh Gambar Dataset . . . . .     | 9        |

## Daftar Gambar

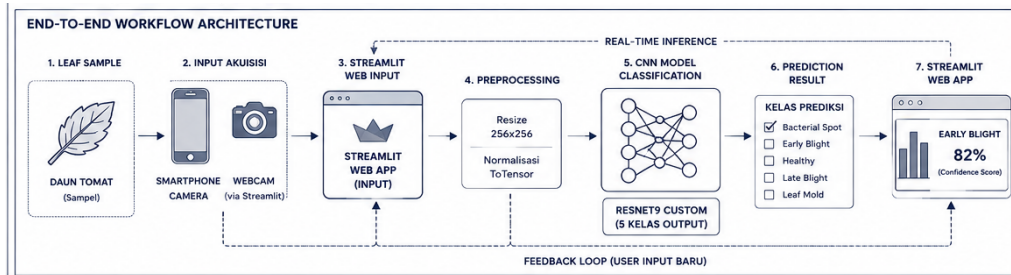
|     |                                                         |    |
|-----|---------------------------------------------------------|----|
| 1.1 | Gambaran Alur Kerja (End-to-End Workflow) . . . . .     | 1  |
| 1.2 | Detail Pipeline Pemrosesan Data dan Pemodelan . . . . . | 1  |
| 2.1 | Contoh gambar daun dari dataset . . . . .               | 10 |

# Bab 1

## Pendahuluan

### 1.1 Gambaran Umum

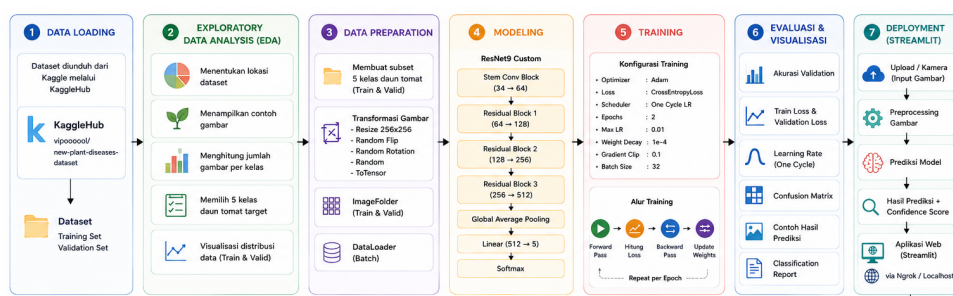
Pada praktikum ini, kita akan membangun sebuah purwarupa (prototype) sistem Machine Learning berbasis Computer Vision yang bertujuan untuk memprediksi secara dini kondisi kesehatan daun tomat. Praktikum ini menggunakan pendekatan end-to-end, mulai dari pemuatan data (*data loading*) hingga fase *deployment* dalam bentuk aplikasi web interaktif.



Gambar 1.1: Gambaran Alur Kerja (End-to-End Workflow)

Sebagaimana diilustrasikan pada Gambar 1.1, alur kerja dalam proyek Machine Learning ini merupakan proses berkesinambungan yang berawal dari pengumpulan dan penyiapan dataset daun tomat, diikuti proses perancangan serta pelatihan model jaringan saraf tiruan. Setelah model dievaluasi dan dianggap memadai, sistem diakhiri dengan proses integrasi (deployment) ke dalam platform aplikasi web interaktif agar model siap digunakan oleh para pengguna akhir.

Sistem ini dirancang tidak untuk memberikan diagnosis final layaknya pandangan pakar agrikultur, melainkan digunakan sebagai alat bantu *screening* awal berdasarkan ciri-ciri visual yang tampak pada gambar daun.



Gambar 1.2: Detail Pipeline Pemrosesan Data dan Pemodelan

Lebih rinci lagi, struktur (pipeline) komputasi di bawah kap mesin dapat divisualisasikan pada Gambar 1.2. Alur komputasi ini memperlihatkan transformasi gambar asli ke dalam matriks angka (Tensor) yang sesuai dengan kerangka kerja PyTorch, yang selanjutnya akan diproses secara matematis oleh lapisan-lapisan (layers) pada model arsitektur ResNet9 guna membedakan pola penyakit pada daun tomat.

## 1.2 Tujuan Praktikum

Adapun tujuan utama dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memberikan pengalaman praktik alur kerja (workflow) pembuatan model Machine Learning secara utuh.
2. Membuat model Machine Learning untuk mengklasifikasikan gambar daun tomat.
3. Menampilkan hasil prediksi model dalam bentuk aplikasi web sederhana.
4. Memahami bahwa model memiliki batasan dan hanya valid untuk data yang sesuai dengan domain pelatihan.

## 1.3 Batasan Sistem

Untuk memastikan fokus pembelajaran tetap terarah, sistem yang akan dibangun memiliki beberapa batasan:

- Dataset difokuskan secara eksklusif pada **5 kelas kondisi daun tomat**:
  1. Bacterial Spot
  2. Early Blight
  3. Late Blight
  4. Leaf Mold
  5. Healthy (Sehat)
- Sistem tidak dapat digunakan untuk memprediksi jenis tanaman lain.
- Model yang dilatih belum dilengkapi dengan kelas penolakan (seperti kelas 'Unknown' atau 'Non Leaf'), sehingga input gambar diasumsikan selalu berupa daun tomat.

## 1.4 Persiapan Lingkungan Praktikum

Pada praktikum ini, spesifikasi komputer lokal yang tinggi tidak diwajibkan karena seluruh proses komputasi, termasuk pelatihan model yang membutuhkan GPU, akan dilakukan di *cloud*. Berikut adalah *tools* dan lingkungan yang perlu dipersiapkan:

Berikut adalah rincian masing-masing teknologi yang menjadi tulang punggung praktikum ini:

- **Google Colab**: Lingkungan Jupyter Notebook gratis berbasis cloud dari Google yang menyediakan akses ke GPU. Digunakan untuk mengeksekusi kode Python, melatih model, dan menjalankan *\*server\** aplikasi.
- **KaggleHub**: Library untuk mengunduh dataset secara langsung dari Kaggle ke environment Google Colab.

- **PyTorch dan Torchvision:** Framework Deep Learning yang akan digunakan untuk mengelola dataset dan membangun arsitektur jaringan saraf tiruan (ResNet9).
- **Streamlit:** Framework open-source berbasis Python untuk membuat antarmuka web interaktif khusus Machine Learning.
- **Ngrok:** Layanan *tunneling* yang memungkinkan aplikasi Streamlit yang berjalan di *localhost* Google Colab dapat diakses melalui internet menggunakan tautan publik.

Pastikan Anda telah memiliki akun Google untuk menggunakan Google Colab. Pada bab selanjutnya, kita akan mulai mempersiapkan *\*environment\** praktikum.

## Bab 2

### Persiapan dan Eksplorasi Data

Bab ini akan memandu Anda dalam mempersiapkan seluruh kebutuhan data yang akan digunakan pada praktikum ini. Terdapat tiga tahapan utama yang akan dilalui, yaitu pengambilan data (*data loading*) dari KaggleHub, persiapan *environment* dengan mengimpor seluruh *library* yang dibutuhkan, serta eksplorasi data awal (*Exploratory Data Analysis* / EDA) untuk memahami karakteristik dataset yang akan digunakan.

#### 2.1 Data Loading dengan KaggleHub

Langkah pertama yang harus dilakukan adalah mengambil dataset gambar daun tanaman dari KaggleHub. Dataset akan digunakan sebagai bahan utama untuk melatih model *Machine Learning* agar mampu mengenali kondisi kesehatan tanaman berdasarkan citra daun.

##### 2.1.1 Pengambilan Data

Tahap awal untuk memulai praktikum ini adalah mendatangkan bahan baku berupa ribuan foto tanaman dari repositori *online* ke dalam lingkungan komputasi kita. Dataset yang digunakan berasal dari Kaggle dengan nama `vipooooool/new-plant-diseases-dataset`.

KaggleHub adalah *library* Python yang memungkinkan kita mengunduh dataset dari Kaggle secara langsung tanpa perlu mengunduh secara manual melalui *browser*. Dengan menggunakan KaggleHub, dataset akan secara otomatis diunduh dan diekstrak ke dalam *path* yang dapat langsung digunakan.

Berikut adalah kode untuk mengunduh dataset:

```
1 import kagglehub
2
3 # Mengunduh dataset Plant Disease dari KaggleHub
4 dataset_path = kagglehub.dataset_download(
5     "vipooooool/new-plant-diseases-dataset"
6 )
7
8 # Menampilkan lokasi dataset yang berhasil diunduh
9 print("Dataset berhasil diunduh.")
10 print("Lokasi dataset:", dataset_path)
```

Listing 2.1: Mengunduh dataset dari KaggleHub

Ketika kode di atas dijalankan, KaggleHub akan mengunduh dataset dan menyimpannya ke dalam direktori *cache* lokal. Variabel `dataset_path` akan menyimpan lokasi lengkap dari dataset yang telah diunduh. Lokasi ini nantinya akan digunakan pada tahap-tahap selanjutnya untuk mengakses gambar-gambar yang akan diproses.

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 Dataset berhasil diunduh.
2 Lokasi dataset: /kaggle/input/new-plant-diseases-dataset
```

Listing 2.2: Output pengunduhan dataset



### Catatan Penting

Pastikan Anda telah terhubung ke internet saat menjalankan kode di atas. Dataset yang diunduh memiliki ukuran yang cukup besar, sehingga proses pengunduhan mungkin memerlukan waktu beberapa saat tergantung pada kecepatan koneksi internet Anda.

#### 2.1.2 Mengecek Struktur Folder Dataset

Setelah dataset berhasil diunduh, langkah berikutnya adalah mengecek isi folder utama dataset. Tahap ini penting karena setiap dataset Kaggle bisa memiliki struktur folder yang berbeda. Dengan melihat isi folder utama, kita dapat menentukan lokasi folder *train*, *valid*, dan *test* yang akan digunakan pada proses pelatihan model.

Berikut adalah kode untuk mengecek struktur folder dataset:

```
1 import os
2
3 # Menampilkan lokasi dataset
4 print("Lokasi dataset:")
5 print(dataset_path)
6
7 # Mengecek isi folder utama dataset
8 print("\nIsi folder utama dataset:")
9
10 for item in os.listdir(dataset_path):
11     item_path = os.path.join(dataset_path, item)
12
13     if os.path.isdir(item_path):
14         print(f"[Folder] {item}")
15     else:
16         print(f"[File] {item}")
```

Listing 2.3: Mengecek struktur folder dataset

Setelah kode di atas dijalankan, akan terlihat struktur folder dataset sebagai berikut:

```
1 Lokasi dataset:
2 /kaggle/input/new-plant-diseases-dataset
3
4 Isi folder utama dataset:
5 [Folder] New Plant Diseases Dataset(Augmented)
6 [Folder] new plant diseases dataset(augmented)
7 [Folder] test
```

Listing 2.4: Output struktur folder dataset

Dari output tersebut, kita dapat melihat bahwa dataset terdiri dari tiga folder utama. Dua di antaranya merupakan folder yang berisi data *training* dan *validation* yang sudah di-*augmentasi*, serta satu folder *test* yang berisi data pengujian. Pada praktikum ini, kita akan menggunakan folder yang berisi data *augmented* untuk melatih dan mengevaluasi model.

## 2.2 Persiapan Environment

Sebelum memulai proses pengolahan data dan pemodelan, kita perlu menyiapkan *environment* dengan mengimpor seluruh *library* yang dibutuhkan. *Library* yang akan digunakan meliputi *library* untuk pengelolaan data, visualisasi, dan *deep learning*.

### 2.2.1 Mengimpor Library

Berikut adalah kode untuk mengimpor seluruh *library* yang dibutuhkan dalam praktikum ini:

```
1 # Library untuk pengelolaan file dan data
2 import os
3 import numpy as np
4 import pandas as pd
5
6 # Library untuk visualisasi
7 import matplotlib.pyplot as plt
8 from PIL import Image
9
10 # Library utama PyTorch
11 import torch
12 import torch.nn as nn
13 import torch.nn.functional as F
14
15 # Library untuk pengolahan dataset dan DataLoader
16 from torch.utils.data import DataLoader
17 from torchvision.datasets import ImageFolder
18 import torchvision.transforms as transforms
19 from torchvision.utils import make_grid
20
21 # Library untuk melihat ringkasan arsitektur model
22 from torchsummary import summary
23
24 # Menampilkan plot langsung di notebook
25 %matplotlib inline
26
27 print("Semua library berhasil diimport.")
```

Listing 2.5: Mengimpor library yang dibutuhkan

Setelah kode di atas dijalankan, akan muncul output berikut:

```
1 Semua library berhasil diimport.
```

Listing 2.6: Output impor library

Penjelasan masing-masing *library* yang diimpor:

- `os` — Digunakan untuk mengelola file dan direktori.
- `numpy` — Library untuk komputasi numerik dan operasi matriks.
- `pandas` — Library untuk manipulasi dan analisis data tabular.
- `matplotlib.pyplot` — Library untuk membuat grafik dan visualisasi data.

- `PIL.Image` — Library untuk membuka dan memproses gambar.
- `torch` — Library utama PyTorch untuk *deep learning*.
- `torch.nn` — Modul PyTorch untuk membangun arsitektur jaringan saraf tiruan.
- `torch.nn.functional` — Modul PyTorch untuk fungsi-fungsi aktivasi dan *loss*.
- `DataLoader` — Digunakan untuk memuat data dalam bentuk *batch*.
- `ImageFolder` — Dataset loader khusus untuk data gambar terstruktur dalam folder.
- `transforms` — Modul untuk melakukan transformasi gambar.
- `make_grid` — Utility untuk menampilkan kumpulan gambar dalam bentuk grid.
- `torchsummary` — Library untuk menampilkan ringkasan arsitektur model.

### 2.2.2 Menginstal Library Tambahan

Beberapa *library* mungkin belum tersedia di *environment* Google Colab secara bawaan. Oleh karena itu, kita perlu menginstalnya terlebih dahulu menggunakan `pip`. Salah satu *library* yang perlu diinstal adalah `torchsummary`:

```
1 !pip install torchsummary -q
2
3 print("Library torchsummary berhasil diinstall.")
```

Listing 2.7: Menginstal library `torchsummary`

Output yang akan tampil:

```
1 Library torchsummary berhasil diinstall.
```

Listing 2.8: Output instalasi `torchsummary`

#### Catatan Penting

Perintah `!pip install` hanya perlu dijalankan satu kali pada awal sesi Google Colab. Jika *runtime* di-reset, Anda perlu menginstal ulang *library* yang tidak termasuk dalam bawaan Colab.

Setelah semua *library* berhasil diimpor dan diinstal, *environment* praktikum siap digunakan untuk tahap selanjutnya, yaitu eksplorasi data awal (*Exploratory Data Analysis*).

### 2.3 Exploratory Data Analysis (EDA)

Setelah dataset berhasil dimuat dan *environment* siap, tahap berikutnya adalah melakukan *Exploratory Data Analysis* (EDA). EDA digunakan untuk memahami kondisi awal dataset, seperti jumlah total gambar, struktur folder, jumlah kelas, dan sebaran data pada setiap kelas. Tahap ini penting agar kita mengetahui apakah dataset sudah layak digunakan untuk proses *training* model *Machine Learning*.

### 2.3.1 Menentukan Lokasi Folder Dataset

Sebelum melakukan EDA, kita perlu menentukan lokasi folder dataset yang akan digunakan. Pada dataset ini, data *training* dan *validation* berada di dalam folder New Plant Diseases Dataset(Augmented). Folder train digunakan untuk melatih model, sedangkan folder valid digunakan untuk mengevaluasi performa model selama proses *training*.

Tahap ini penting karena variabel `train_dir`, `valid_dir`, dan `diseases` akan digunakan pada proses EDA dan *preprocessing* berikutnya.

Berikut adalah kode untuk menentukan lokasi folder dataset:

```
1 import os
2
3 # Path ke folder dataset utama
4 dataset_main = os.path.join(
5     dataset_path,
6     "New Plant Diseases Dataset(Augmented)",
7     "New Plant Diseases Dataset(Augmented)"
8 )
9
10 # Path ke folder training dan validation
11 train_dir = os.path.join(dataset_main, "train")
12 valid_dir = os.path.join(dataset_main, "valid")
13
14 # Menampilkan lokasi folder
15 print("Folder dataset utama :", dataset_main)
16 print("Folder training      :", train_dir)
17 print("Folder validation     :", valid_dir)
18
19 # Mendapatkan daftar kelas (nama folder dalam train)
20 diseases = os.listdir(train_dir)
21 print("Jumlah kelas dataset :", len(diseases))
```

Listing 2.9: Menentukan lokasi folder dataset

Output yang akan tampil:

```
1 Folder dataset utama : /kaggle/input/new-plant-diseases-dataset/New
   Plant
2                               Diseases Dataset(Augmented)/New Plant Diseases
3                               Dataset(Augmented)
4 Folder training      : /kaggle/input/new-plant-diseases-dataset/New
   Plant
5                               Diseases Dataset(Augmented)/New Plant Diseases
6                               Dataset(Augmented)/train
7 Folder validation     : /kaggle/input/new-plant-diseases-dataset/New
   Plant
8                               Diseases Dataset(Augmented)/New Plant Diseases
9                               Dataset(Augmented)/valid
10 Jumlah kelas dataset : 38
```

Listing 2.10: Output lokasi folder dataset

Dari output di atas, kita dapat melihat bahwa dataset utama memiliki total 38 kelas penyakit tanaman. Namun, untuk praktikum ini kita hanya akan fokus pada 5 kelas daun tomat.

### 2.3.2 Menghitung Total Gambar dalam Dataset

Setelah mengetahui struktur folder, langkah selanjutnya adalah menghitung total seluruh gambar yang tersedia untuk memastikan kita memiliki cukup banyak contoh foto untuk dipelajari oleh model.

Berikut adalah kode untuk menghitung total gambar dalam dataset:

```
1 # Ekstensi file gambar yang akan dihitung
2 image_extensions = (".jpg", ".jpeg", ".png")
3
4 # Variabel untuk menyimpan jumlah total gambar
5 total_images = 0
6
7 # Menelusuri seluruh folder dan subfolder dataset
8 for root, dirs, files in os.walk(dataset_path):
9     image_files = [
10         file for file in files
11         if file.lower().endswith(image_extensions)
12     ]
13
14     total_images += len(image_files)
15
16 # Menampilkan hasil
17 print("Total gambar dalam dataset:", total_images)
```

Listing 2.11: Menghitung total gambar dalam dataset

Setelah kode dijalankan, akan tampil output berikut:

```
1 Total gambar dalam dataset: 175767
```

Listing 2.12: Output total gambar dalam dataset

Kode di atas akan menelusuri seluruh folder dan subfolder dalam dataset, kemudian menghitung jumlah file gambar dengan ekstensi .jpg, .jpeg, dan .png. Hasilnya menampilkan total seluruh gambar yang tersedia di dalam dataset, yaitu sebanyak **175.767** gambar.

### 2.3.3 Menampilkan Contoh Gambar Dataset

Setelah mengetahui jumlah total gambar, langkah selanjutnya adalah melihat contoh gambar dari dataset. Hal ini penting untuk memahami karakteristik visual dari data yang akan diproses, seperti ukuran gambar, kualitas resolusi, dan variasi kondisi daun yang ada.

Berikut adalah kode untuk menampilkan contoh gambar dari dataset:

```
1 # Ekstensi gambar yang akan dicari
2 image_extensions = (".jpg", ".jpeg", ".png")
3
4 # Menyimpan seluruh path gambar
5 image_paths = []
6
7 # Menelusuri seluruh folder dataset
8 for root, dirs, files in os.walk(dataset_path):
9     for file in files:
10         if file.lower().endswith(image_extensions):
```

```

11         image_paths.append(os.path.join(root, file))
12
13 # Menampilkan jumlah gambar yang ditemukan
14 print("Jumlah gambar ditemukan:", len(image_paths))
15
16 # Mengecek apakah ada gambar yang ditemukan
17 if len(image_paths) == 0:
18     print("Tidak ada gambar yang ditemukan. Periksa kembali lokasi dataset.")
19 else:
20     # Mengambil gambar pertama sebagai contoh
21     sample_image_path = image_paths[0]
22
23     # Membuka gambar
24     img = Image.open(sample_image_path).convert("RGB")
25
26     # Menampilkan informasi gambar
27     print("Contoh path gambar:")
28     print(sample_image_path)
29     print("Ukuran gambar:", img.size)
30
31     # Menampilkan gambar
32     plt.figure(figsize=(5, 5))
33     plt.imshow(img)
34     plt.axis("off")
35     plt.title("Contoh Gambar Dataset")
36     plt.show()

```

Listing 2.13: Menampilkan contoh gambar dataset

Output yang akan tampil setelah kode dijalankan:

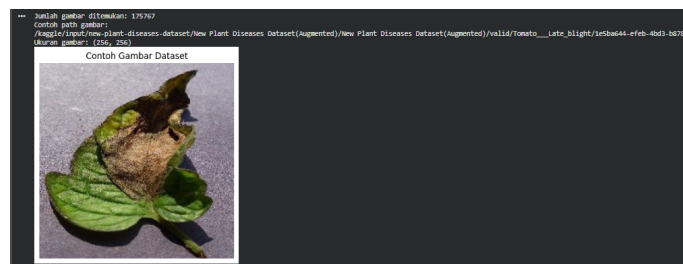
```

1 Jumlah gambar ditemukan: 175767
2 Contoh path gambar:
3 /kaggle/input/new-plant-diseases-dataset/New Plant
4 Diseases Dataset (Augmented)/New Plant Diseases
5 Dataset (Augmented)/valid/Tomato___Late_blight/1e5ba644-
6 efeb-4bd3-b878-a0606cf8a992___RS_Late.B 6272_flipLR.JPG
7 Ukuran gambar: (256, 256)

```

Listing 2.14: Output teks contoh gambar dataset

Selain informasi di atas, kode tersebut juga akan menampilkan visualisasi gambar secara langsung. Gambar 2.1 berikut adalah contoh gambar yang dihasilkan:



Gambar 2.1: Contoh gambar daun dari dataset

Dari output di atas, kita dapat melihat bahwa setiap gambar dalam dataset memiliki

ukuran **256×256** piksel. Gambar contoh yang ditampilkan berasal dari kelas `Tomato___Late_blight` (penyakit hawar daun tomat). Selain informasi path dan ukuran, kode di atas juga akan menampilkan visualisasi gambar secara langsung di *notebook*.

#### Catatan Penting

Pada lingkungan Google Colab, perintah `plt.show()` akan menampilkan gambar langsung di bawah sel kode. Jika Anda menjalankan kode di *terminal* biasa, pastikan *backend* matplotlib telah dikonfigurasi dengan benar untuk menampilkan grafik.